

Projet NetVision

Supervision radio et aide à la décision pour réseau mobile

NetVision Digital Twin : tableau de bord de supervision RAN, d'analyse QoS et de simulation opérationnelle

Rapport technique basé sur la solution exécutée localement

Périmètre	Application locale NetVision inspectée et exécutée sur le poste de travail
Dépôt analysé	odc-rami-ui-refactor, branche locale principale
Stack principale	Next.js, React, MapLibre, API routes Next.js, FastAPI optionnel, Redis/BullMQ, ns-3 via WSL
Mode de données capturé	mock, avec distinction explicite vis-à-vis du mode réel
Date de validation	24 mai 2026

Rapport rédigé en français à partir du code source, des API locales et de captures réelles du tableau de bord.

Remerciements

Ce rapport synthétise le travail réalisé autour de NetVision, une solution locale d'observation et d'aide à la décision pour un réseau radio mobile. Il tient compte de l'état réel de l'application inspectée, des services disponibles au moment de la validation, ainsi que des limites encore présentes. L'objectif n'est pas de présenter une vision idéalisée du produit, mais de documenter fidèlement ce qui fonctionne, ce qui est optionnel et ce qui reste à renforcer.

Résumé

NetVision est une application web locale destinée à aider un ingénieur radio à surveiller un réseau mobile, identifier les zones de congestion, analyser les indicateurs de qualité radio et lancer des simulations opérationnelles sur une cellule sélectionnée. La solution repose sur une interface Next.js/React, une carte MapLibre de la Tunisie, des données runtime structurées en fichiers JSON, des routes API Next.js, un backend FastAPI optionnel pour les recommandations, ainsi qu'une file de jobs Redis/BullMQ pour les simulations asynchrones.

La version inspectée met l'accent sur un parcours opérateur : vue réseau, heures critiques, qualité radio et action cellule. Les outils d'administration, d'import/export, de qualité des données et d'état système sont accessibles en mode administrateur. La simulation ns-3 est intégrée comme moteur asynchrone principal pour les travaux de type *what-if*. Elle ne doit pas être interprétée comme un jumeau radio parfaitement calibré : les résultats sont encadrés par des garde-fous, des hypothèses et une couche de crédibilité. Les captures insérées dans ce rapport proviennent de l'application exécutée localement.

Synthèse technique

Le projet fournit un socle opérationnel solide pour l'analyse RAN locale : chargement de données, agrégation par gouvernorat, délégation, site et cellule, visualisation cartographique, chronologie horaire, tableaux KPI, diagnostics multi-indicateurs et simulation asynchrone. Les services FastAPI, Redis/BullMQ et ns-3 sont présents et vérifiés dans l'environnement local utilisé pour ce rapport. Certaines briques restent toutefois à considérer comme expérimentales ou perfectibles : calibration radio avancée, fidélité physique ns-3, industrialisation de l'import, authentification de production et qualification exhaustive en conditions réelles.

Liste des acronymes

Acronyme	Signification
API	Interface de programmation applicative
CQI	Channel Quality Indicator, indicateur de qualité radio
EPC	Evolved Packet Core
KPI	Indicateur clé de performance
LTE	Long Term Evolution
NOC	Centre d'exploitation réseau
PRB	Physical Resource Block
QoE	Qualité d'expérience perçue par l'utilisateur
QoS	Qualité de service mesurée par le réseau
RAN	Radio Access Network
Redis	Base mémoire utilisée ici pour la file de jobs BullMQ
TA	Timing Advance
UE	User Equipment, terminal utilisateur
WSL	Windows Subsystem for Linux

Table des figures

6.1	Vue principale du tableau de bord NetVision au chargement.	11
6.2	Vue réseau nationale avec carte, légende et synthèse KPI.	12
6.3	Onglet Heures critiques et analyse des fenêtres de charge.	12
6.4	Sélection de la cellule TN1158_c01 et panel Qualité radio.	13
6.5	Espace Action cellule avec recommandation et paramètres de simulation. .	13
6.6	Mode administrateur activé par ?admin=1.	14
6.7	Panel Données : qualité runtime, import et export.	14
6.8	Panel Admin : état des services et diagnostics système.	15

Liste des tableaux

4.1	Besoins fonctionnels couverts par NetVision	6
4.2	État des jeux runtime inspectés	8
6.1	Synthèse de validation	16

Table des matières

Remerciements	i
Résumé	ii
Synthèse technique	iii
Liste des acronymes	iv
1 Introduction générale	1
2 Contexte général et problématique	2
2.1 Contexte du projet	2
2.2 Problématique	2
2.3 Objectifs	3
2.4 Solution proposée	3
2.5 Périmètre réel et limites	3
3 Fondements théoriques RAN	4
3.1 Cellule, site et secteur	4
3.2 PRB Load	4
3.3 Débit	4
3.4 CQI	4
3.5 Utilisateurs actifs et Timing Advance	5
3.6 Heure critique et congestion récurrente	5
3.7 Diagnostic multi-KPI	5
4 Spécification et conception	6
4.1 Acteurs	6
4.2 Besoins fonctionnels	6

4.3	Besoins non fonctionnels	7
4.4	Architecture globale	7
4.5	Architecture frontend	7
4.6	Architecture backend	7
4.7	Modèle de données	7
5	Réalisation technique	9
5.1	Chargement et agrégation des données	9
5.2	Carte et navigation	9
5.3	Panels opérateur	9
5.4	Import, export et administration	9
5.5	Recommandation et simulation	10
5.6	Solution de robustesse et de fiabilité	10
6	Résultats et validation	11
6.1	Captures de l'interface	11
6.2	Validation technique	15
6.3	Limites observées pendant la validation	16
7	Gestion du projet	17
7.1	Méthodologie	17
7.2	Phases de travail	17
7.3	Difficultés rencontrées	17
7.4	Décisions techniques	18
8	Limites et perspectives	19
8.1	Limites actuelles	19
8.2	Perspectives	19
9	Conclusion générale	20
A	Commandes de reproduction	21
B	Fichiers principaux inspectés	22

Introduction générale

La supervision d'un réseau radio mobile exige de croiser plusieurs familles de données : charge radio, débit, nombre d'utilisateurs, qualité de lien, localisation des cellules, évolution temporelle et récurrence des congestions. Une cellule avec un PRB élevé n'est pas nécessairement problématique si le débit et la qualité radio restent corrects. À l'inverse, un PRB élevé accompagné d'un débit faible peut signaler une pression capacitaire, tandis qu'un débit faible avec un CQI faible oriente davantage vers un problème de qualité radio, de couverture ou d'interférence.

NetVision répond à ce besoin par une interface locale orientée ingénieur radio. Elle fournit une lecture nationale et régionale du réseau, une analyse des heures critiques, une inspection QoS au niveau cellule, puis un espace d'action permettant de simuler des corrections possibles. Le produit observé n'est pas un simple tableau de démonstration : il inclut des routes API, un modèle de données runtime, des contrôles de robustesse, un mode administrateur et une file de jobs pour les simulations.

Ce rapport décrit la solution réellement présente dans le dépôt et vérifiée localement. Les fonctionnalités non finalisées ou dépendantes de services optionnels sont signalées explicitement.

Contexte général et problématique

2.1 Contexte du projet

Les réseaux mobiles tunisiens couvrent un territoire hétérogène composé de zones urbaines denses, de zones périurbaines et de délégations à charge plus variable. Dans ce contexte, la supervision RAN ne peut pas se limiter à des courbes KPI isolées. Elle doit permettre de localiser les zones affectées, de comprendre les causes probables et de prioriser les actions techniques.

NetVision vise ce type d'usage. L'application combine une carte administrative, des cellules radio géolocalisées, une chronologie horaire et des panels d'analyse. Elle est pensée pour un usage local-first : le tableau de bord continue à fonctionner sur les données runtime même si certains services optionnels sont indisponibles.

2.2 Problématique

L'ingénieur radio doit répondre rapidement à quatre questions :

1. Où se situent les congestions ou les zones sous surveillance ?
2. Les problèmes sont-ils ponctuels ou liés à des heures critiques récurrentes ?
3. Les KPI indiquent-ils une pression capacitaire, une qualité radio dégradée ou une charge acceptable ?
4. Quelle action opérationnelle peut être testée avant décision ?

Sans un outil intégré, ces questions nécessitent souvent plusieurs exports, cartes et scripts. NetVision cherche à réduire cette fragmentation.

2.3 Objectifs

Les objectifs fonctionnels constatés sont les suivants :

- visualiser l'état réseau à l'échelle nationale, gouvernorat, délégation et cellule ;
- suivre les tranches horaires et les variations de charge ;
- afficher des KPI pratiques : PRB, débit, CQI, utilisateurs actifs, TA et santé ;
- rechercher et sélectionner une cellule comme TN1158_c01 ;
- accéder au parcours qualité radio puis action cellule ;
- lancer des simulations asynchrones lorsque les préconditions sont satisfaites ;
- réserver les outils de données, diagnostics et exports au mode administrateur.

2.4 Solution proposée

La solution proposée est une application web Next.js exécutée localement. Le coeur de l'expérience est le cockpit opérateur. Les données sont lues depuis `runtime_data` ou `runtime_data_mock`, normalisées côté frontend, puis agrégées dans des panels et une carte MapLibre.

Les services complémentaires sont séparés :

- FastAPI fournit un moteur de recommandations déterministes et des exports ;
- Redis/BullMQ gère les jobs asynchrones ;
- ns-3 via WSL fournit un moteur expérimental de simulation *what-if*.

2.5 Périmètre réel et limites

Le périmètre réel couvre la supervision, la lecture QoS, l'import/export administrateur, les contrôles de santé et la simulation asynchrone. Les limites principales sont la dépendance à la qualité des données d'entrée, le caractère encore approximatif des simulations radio, l'absence d'authentification de production complète dans l'environnement local et la nécessité de calibrer davantage les résultats ns-3 avant tout usage décisionnel strict.

Fondements théoriques RAN

3.1 Cellule, site et secteur

Un site radio regroupe généralement plusieurs secteurs. Chaque secteur correspond à une cellule logique orientée par un azimut, une bande de fréquence et des paramètres radio. Dans NetVision, l'entité centrale d'action est la cellule, car elle possède un nom, une localisation, un site, une bande, un azimut et des KPI observés.

3.2 PRB Load

Le PRB Load mesure l'utilisation des ressources radio disponibles. Un PRB élevé indique une charge importante, mais il ne suffit pas à conclure à une mauvaise QoS. Il doit être croisé avec le débit, le CQI et le nombre d'utilisateurs.

3.3 Débit

Le débit reflète la capacité utile perçue. Un débit faible avec un PRB élevé et un CQI acceptable oriente vers une pression capacitaire. Un débit faible avec un CQI faible suggère plutôt une qualité radio dégradée, une couverture insuffisante ou une interférence.

3.4 CQI

Le CQI indique la qualité du lien radio. Un CQI faible réduit l'efficacité spectrale et peut dégrader le débit même lorsque la charge n'est pas extrême. NetVision utilise le CQI comme indice de diagnostic radio, pas comme unique verdict.

3.5 Utilisateurs actifs et Timing Advance

Les utilisateurs actifs renseignent la demande instantanée. Le Timing Advance aide à interpréter la distance approximative des terminaux et peut contribuer à détecter des effets de bord de cellule ou de couverture étendue.

3.6 Heure critique et congestion récurrente

Une heure critique correspond à une période où la charge, le nombre d'utilisateurs ou le taux de congestion atteignent un maximum. Une congestion récurrente aux heures de pointe indique un problème plus structurel qu'un incident isolé.

3.7 Diagnostic multi-KPI

Le diagnostic retenu par NetVision suit une logique multi-indicateurs :

- PRB élevé, débit faible, CQI acceptable : congestion capacitaire probable ;
- PRB élevé, débit faible, CQI faible : qualité radio ou interférence probable ;
- PRB élevé, débit et CQI corrects : charge élevée mais acceptable ;
- données absentes ou incohérentes : diagnostic insuffisant.

Spécification et conception

4.1 Acteurs

Le premier acteur est l'ingénieur radio ou NOC qui consulte l'état réseau, sélectionne une cellule et évalue une action. Le second acteur est l'administrateur local qui gère le mode de données, l'import, l'export, les diagnostics système et la qualité du référentiel.

4.2 Besoins fonctionnels

Les besoins fonctionnels sont synthétisés dans le tableau 4.1.

TABLE 4.1 – Besoins fonctionnels couverts par NetVision

Besoin	Réalisation observée
Vue réseau	Carte nationale, agrégats, tableaux de priorisation et légende de congestion.
Heures critiques	Panel dédié avec profil 24h et zones à traiter selon la métrique sélectionnée.
Qualité radio	KPI cellule, diagnostic multi-KPI et contexte site/délégation.
Action cellule	Recommandation, paramètres d'action, fidélité de simulation et résultat avant/après.
Administration	Mode <code>?admin=1</code> , qualité des données, import dry-run, exports et état système.

4.3 Besoins non fonctionnels

Les besoins non fonctionnels retenus sont la lisibilité opérateur, la disponibilité locale, la séparation entre services core et services optionnels, la validation des entrées, l'absence de fallback silencieux pour la simulation et la traçabilité des erreurs.

4.4 Architecture globale

L'architecture peut être résumée ainsi :

Navigateur

- > Next.js / React
- > API routes Next.js
- > runtime_data ou runtime_data_mock
- > FastAPI optionnel pour recommandations
- > Redis/BullMQ + worker pour jobs
- > ns-3 via WSL pour simulation asynchrone

4.5 Architecture frontend

Le frontend est porté par `pages/index.js`, qui charge dynamiquement `src/main.js`. Le cockpit assemble plusieurs composants : en-tête, rail d'onglets, fil d'ariane, chronologie, carte MapLibre et panels métier. Les onglets opérateur sont : Vue réseau, Heures critiques, Qualité radio et Action cellule. Les onglets Données et Admin sont visibles uniquement en mode administrateur.

4.6 Architecture backend

Les routes API Next.js constituent la couche backend principale du tableau de bord. Elles servent les données runtime, l'état des jobs, les exports, l'import dry-run, les profils d'import et l'état des services. Le backend FastAPI est présent comme service optionnel de recommandations et d'exports spécialisés.

4.7 Modèle de données

Le répertoire `runtime_data` contient le référentiel réel local : `baseline.json`, `time_index.json`, fichiers horaires dans `time_data`, index administratif, graphes de

voisinage et profils d'heures critiques. Le répertoire `runtime_data_mock` contient un jeu plus large utilisé pour les démonstrations et la QA.

TABLE 4.2 – État des jeux runtime inspectés

Jeu	Cellules	Tranches	Délégations	Observation
<code>runtime_data</code>	792	24	263	Données réelles locales sur une journée horaire.
<code>runtime_data_mock</code>	2 640	720	263	Données mock sur 30 jours horaires, utilisées pour les captures.

Réalisation technique

5.1 Chargement et agrégation des données

Le chargement initial lit les fichiers runtime via `/api/data/*`. Les cellules sont normalisées puis agrégées par gouvernorat, délégation et site. La chronologie charge les tranches horaires à la demande et met en cache les slices voisines afin de limiter les rechargements.

5.2 Carte et navigation

La carte utilise MapLibre avec des couches administratives et des points cellules. L'état de carte est dérivé du périmètre courant, de la métrique et des options d'affichage. Les interactions permettent de passer du national vers un gouvernorat, une délégation ou une cellule. En cas de problème WebGL, la solution prévoit une dégradation vers une lecture par tableau.

5.3 Panels opérateur

Le panel Vue réseau donne une synthèse nationale ou régionale. Le panel Heures critiques montre les fenêtres de charge. Le panel Qualité radio détaille les KPI d'une cellule. Le panel Action cellule propose une simulation asynchrone si la file de jobs, les données et le moteur sont disponibles.

5.4 Import, export et administration

Le mode administrateur expose les fonctions de qualité des données, d'import dry-run, de profils d'import, de restauration runtime et d'export. Les exports incluent des métadon-

nées d'audit comme le périmètre, le mode de données, la fenêtre temporelle et la date de génération.

5.5 Recommandation et simulation

Le moteur de recommandations est déterministe et fondé sur des règles. Il ne s'agit pas d'un modèle d'apprentissage automatique opaque. Les actions simulables alignées sur le périmètre actuel sont : ajustement d'inclinaison, rééquilibrage de charge, optimisation des voisins, ajout de porteuse et ajout de secteur.

La simulation ns-3 est intégrée comme moteur principal des jobs asynchrones. Le flux réel est :

```
UI -> /api/jobs -> Redis/BullMQ -> jobWorker.js
    -> ns3JobAdapter.js -> build_scenario.mjs
    -> WSL Ubuntu -> netvision-ran-sim
    -> metrics.json -> ns3ResultAdapter.js -> resultat NetVision
```

Cette simulation reste un modèle *what-if* V1. Elle compare un état avant/après dans un scénario local borné : cellule cible, voisinage, utilisateurs synthétiques et demande inférée. Les sorties sont rejetées si elles violent des garde-fous de plausibilité. La calibration avancée reste une perspective.

5.6 Solution de robustesse et de fiabilité

La robustesse du système repose sur plusieurs choix :

- architecture local-first : le coeur du dashboard fonctionne avec les fichiers runtime ;
- contrats explicites : `baseline.json`, `time_index.json`, fichiers horaires et index administratif ;
- séparation core/optionnel : FastAPI, Redis/BullMQ et ns-3 sont vérifiés mais isolés du chargement principal ;
- health checks : `/api/backend-health` et `/api/jobs-health` exposent l'état des services ;
- garde-fous jobs : validation des actions, du moteur, des fichiers horaires et des pré-conditions cellule/action ;
- qualité des données : rapprochement administratif, cellules appariées, ratio KPI et avertissements ;
- distinction mock/réel : l'API `/api/data-mode` indique le mode actif ;
- absence de fallback silencieux : une simulation ns-3 indisponible doit être signalée explicitement.

Résultats et validation

6.1 Captures de l'interface

Les figures suivantes proviennent de l'application exécutée localement sur `http://127.0.0.1:3000`. Le mode actif pendant la capture était `mock`.

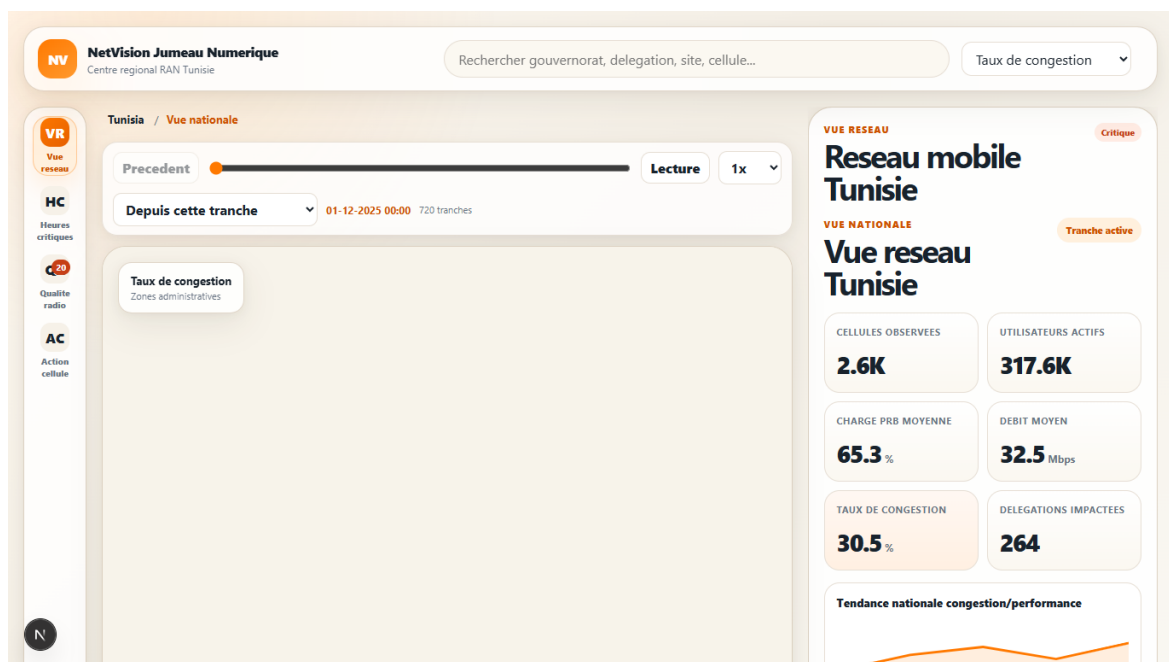


FIGURE 6.1 – Vue principale du tableau de bord NetVision au chargement.

La figure 6.1 montre le cockpit opérateur avec les onglets principaux, la carte et les premiers indicateurs synthétiques.

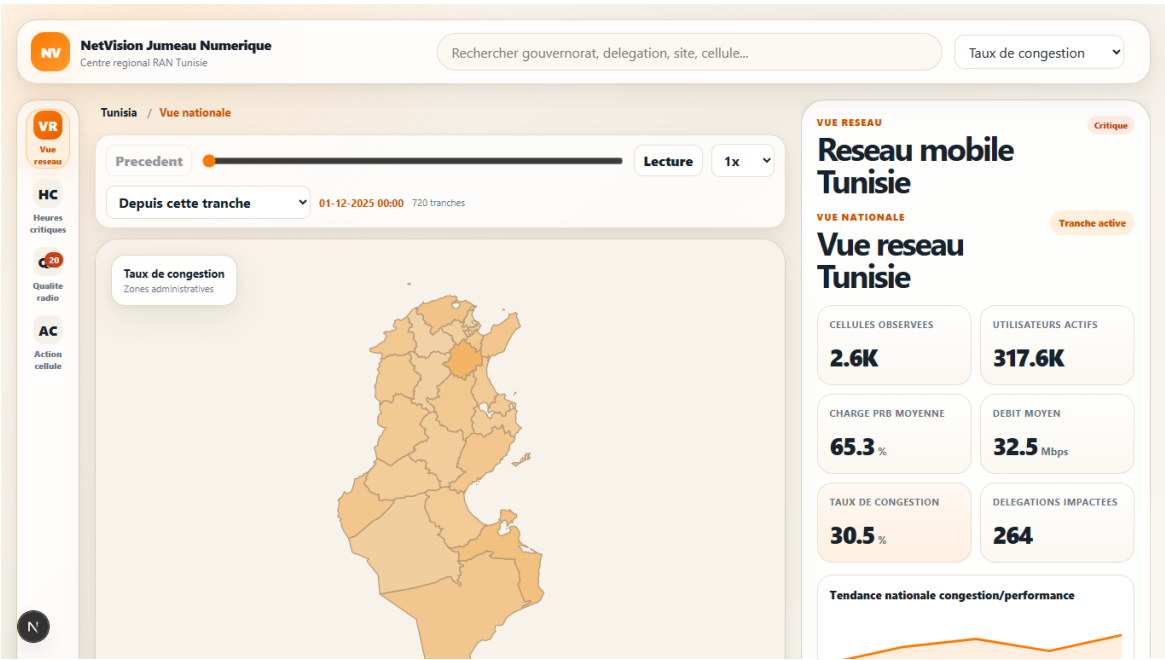


FIGURE 6.2 – Vue réseau nationale avec carte, légende et synthèse KPI.

La figure 6.2 confirme la présence de la carte de Tunisie, des états radio et de la lecture nationale.

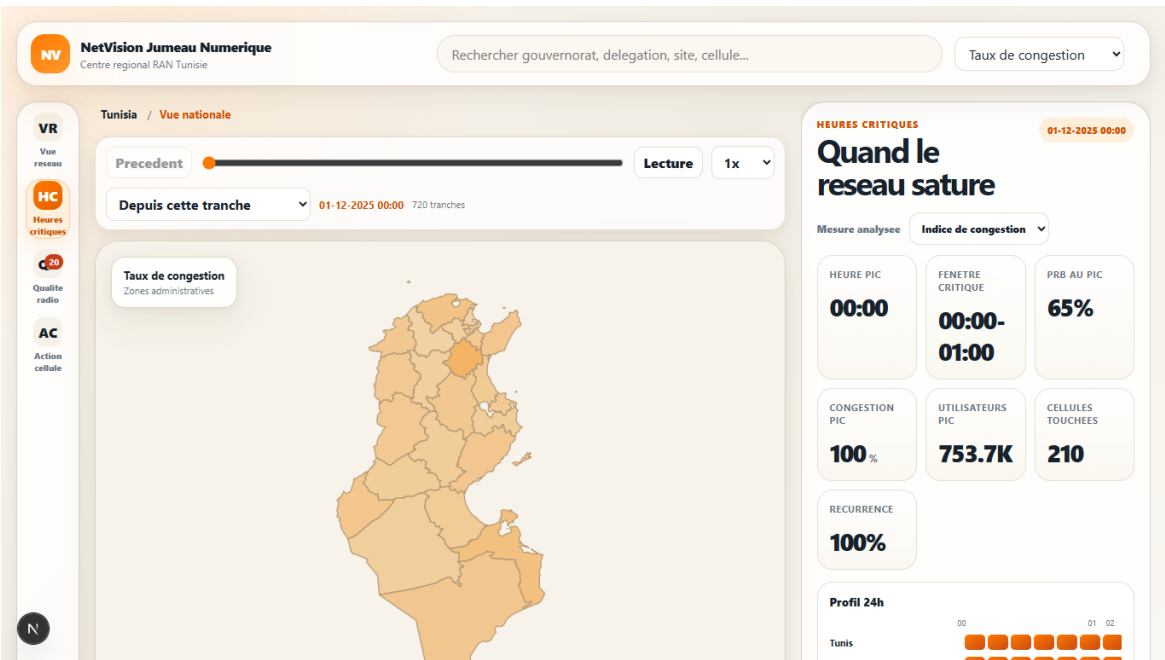


FIGURE 6.3 – Onglet Heures critiques et analyse des fenêtres de charge.

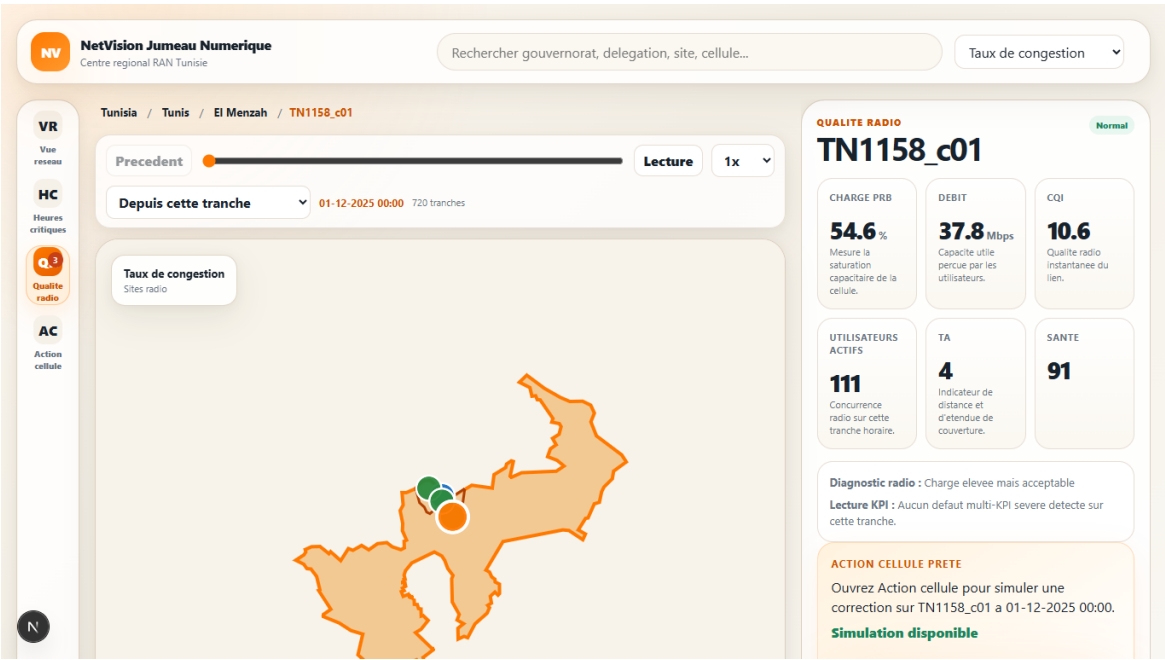


FIGURE 6.4 – Sélection de la cellule TN1158_c01 et panel Qualité radio.

La figure 6.4 illustre le parcours voulu : recherche cellule, périmètre cellule, KPI et diagnostic pratique.

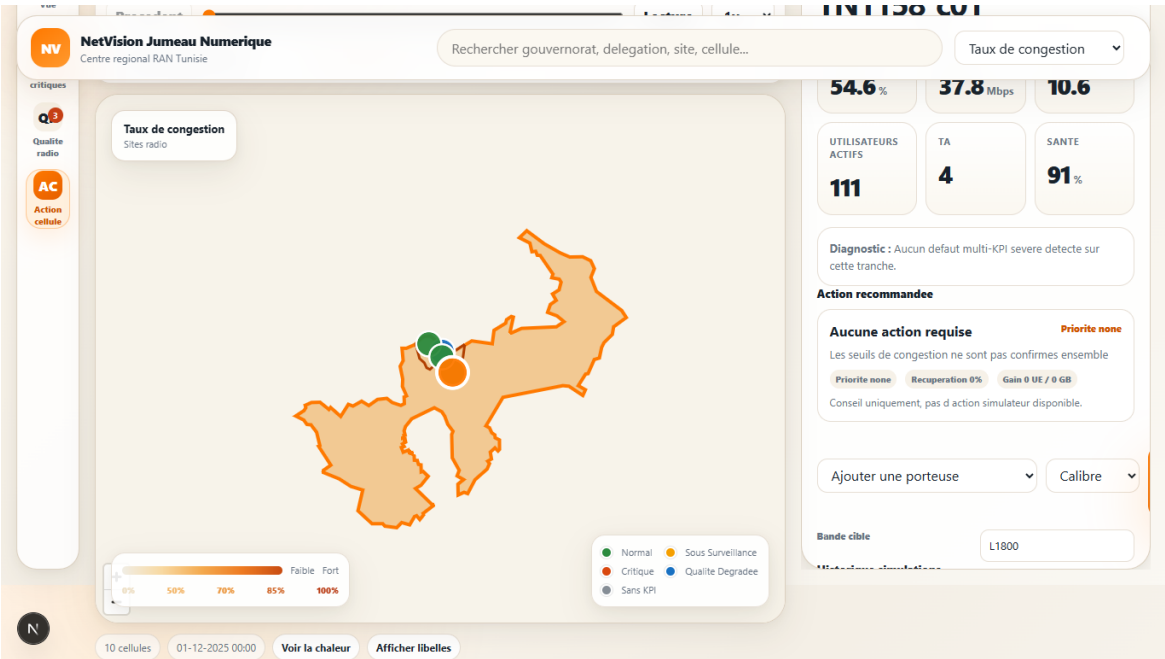


FIGURE 6.5 – Espace Action cellule avec recommandation et paramètres de simulation.

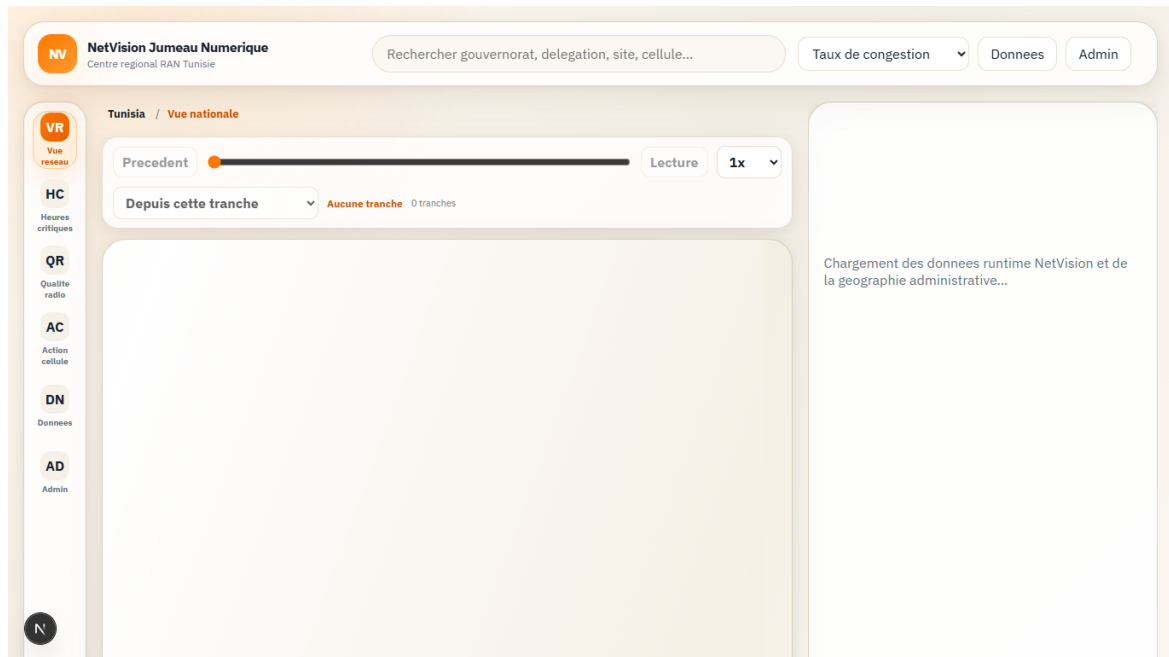


FIGURE 6.6 – Mode administrateur activé par ?admin=1.

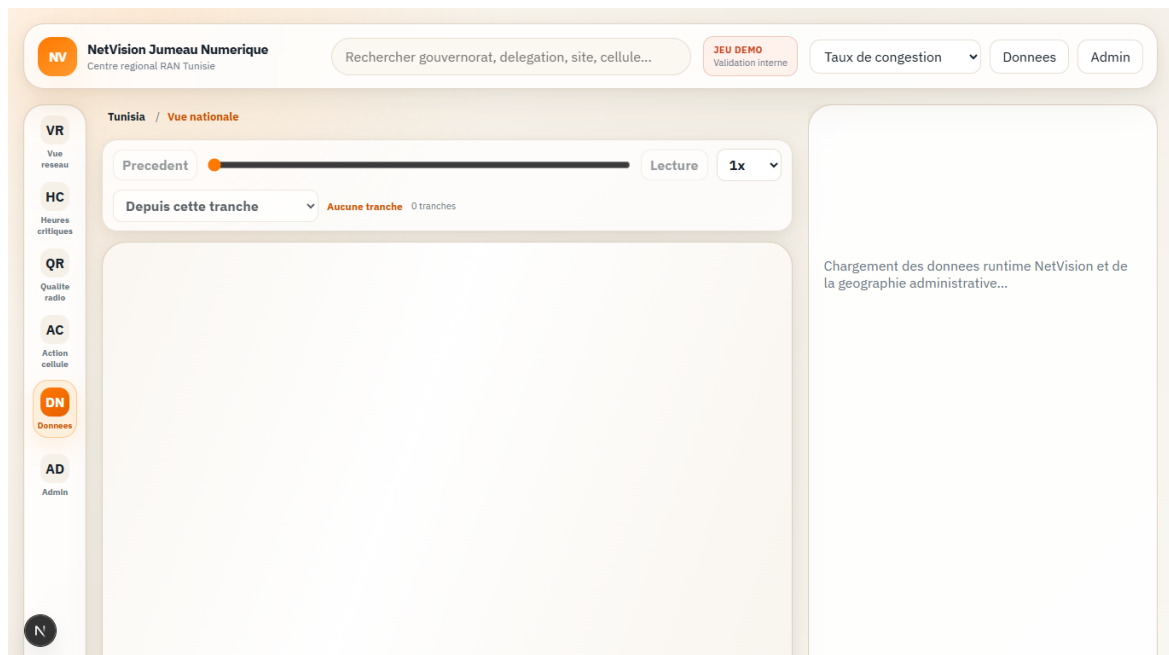


FIGURE 6.7 – Panel Données : qualité runtime, import et export.

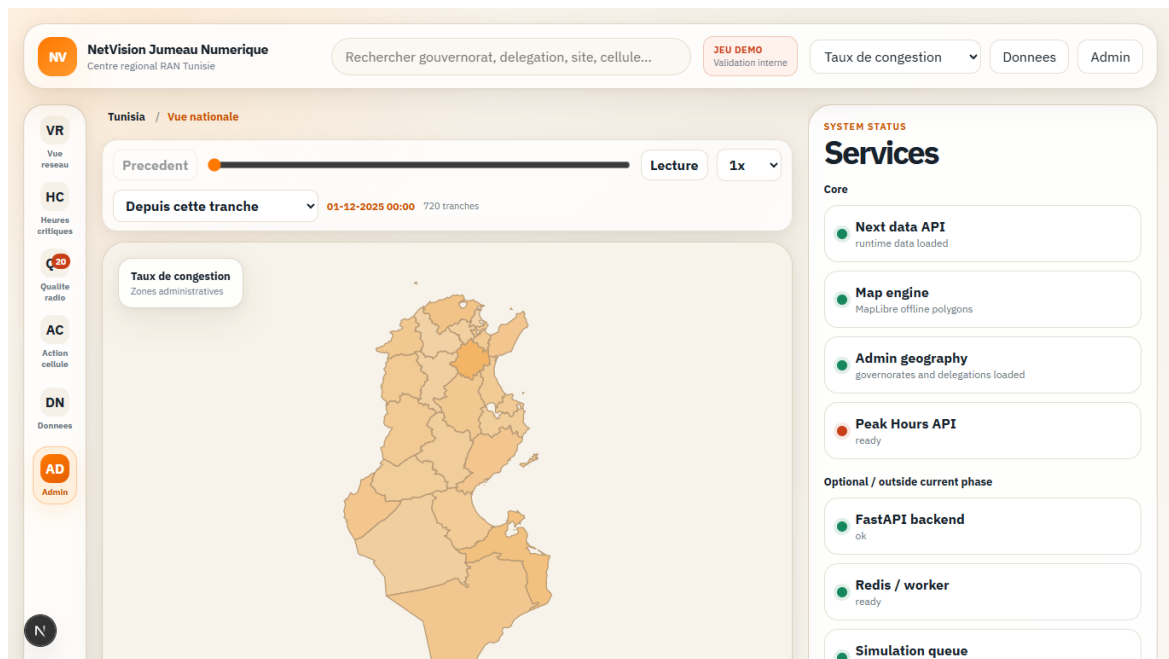


FIGURE 6.8 – Panel Admin : état des services et diagnostics système.

6.2 Validation technique

Les contrôles suivants ont été exécutés localement au moment de la rédaction.

TABLE 6.1 – Synthèse de validation

Contrôle	Résultat	Commentaire
Chargement dashboard	Réussi	<code>http://127.0.0.1:3000/</code> répond avec code 200.
Mode de données	Réussi	<code>/api/data-mode</code> indique <code>mock</code> .
Backend FastAPI	Réussi	<code>/api/backend-health</code> signale le backend disponible avec 792 cellules chargées.
Redis	Réussi	<code>redis-cli</code> répond PONG sur les ports 6381 et 6379.
File simulation/ns-3	Réussi	<code>/api/jobs-health</code> indique Redis et ns-3 prêts.
Contrats API	Réussi	<code>npm run test:contracts</code> passe avec 33 tests.
Smoke core	Réussi	<code>npm run smoke:v2:core</code> passe avec 6 contrôles sur 6.
Smoke simulation	Réussi	<code>npm run smoke:v2:sim</code> passe avec 4 contrôles sur 4, incluant les cinq actions ns-3.
Prérequis ns-3	Réussi	<code>npm run ns3:check</code> passe avec 7 contrôles sur 7.
Build production	Réussi	<code>npm run build</code> compile l'application Next.js.
Captures navigateur	Réussi	8 captures réelles enregistrées dans le rapport.

6.3 Limites observées pendant la validation

La capture a été faite en mode `mock`, ce qui est adapté à la démonstration et aux tests d'ergonomie. Le mode réel existe avec 792 cellules et 24 tranches horaires, mais les figures du rapport reflètent le mode actif dans l'application. Les simulations ns-3 étaient disponibles dans l'environnement local ; néanmoins, leur précision radio dépend encore de la qualité du scénario, des métadonnées antenne et de la calibration.

Gestion du projet

7.1 Méthodologie

Le projet a été conduit par itérations : clarification du parcours opérateur, nettoyage de l'interface, séparation admin/opérateur, renforcement des contrats API, migration progressive vers ns-3 et ajout de contrôles de robustesse.

7.2 Phases de travail

1. Structuration du cockpit autour du parcours ingénieur radio.
2. Normalisation des données runtime et de l'index administratif.
3. Amélioration de la carte et de la chronologie.
4. Ajout de la file de jobs Redis/BullMQ et des validations d'entrée.
5. Intégration ns-3 comme moteur asynchrone.
6. Ajout des panels administrateur, import/export et santé système.
7. Validation par tests de contrats, build et navigation réelle.

7.3 Difficultés rencontrées

Les principales difficultés concernent l'alignement entre les données radio et les frontières administratives, la gestion de la performance en lecture cartographique, la fiabilité des jobs asynchrones, la distinction entre fonctionnalités démonstratives et fonctionnalités réellement exploitables, ainsi que la tentation de présenter la simulation comme plus précise qu'elle ne l'est réellement.

7.4 Décisions techniques

Les décisions importantes sont les suivantes : garder une architecture locale sans Docker obligatoire, isoler FastAPI comme service optionnel, utiliser Redis/BullMQ pour les simulations longues, conserver les API Next.js comme façade principale, rendre les outils techniques visibles seulement en mode administrateur et documenter les limites de ns-3 comme un moteur *what-if* à calibrer.

Limites et perspectives

8.1 Limites actuelles

La solution dépend de fichiers runtime correctement générés. Les données antenne avancées, comme les hauteurs exactes, diagrammes réels, tilt effectif et paramètres de handover, ne sont pas toujours disponibles. Les simulations ns-3 utilisent donc des hypothèses et des utilisateurs synthétiques. L'import CSV est présent mais doit encore être durci pour un usage production complet. L'authentification locale est volontairement légère dans l'environnement de développement.

8.2 Perspectives

Les perspectives les plus pertinentes sont :

- améliorer la calibration ns-3 par zone, bande et action ;
- enrichir les profils busy-hour et les diagnostics de récurrence ;
- ajouter des rapports PDF générés directement depuis l'application ;
- renforcer les tests Playwright de parcours opérateur ;
- industrialiser l'authentification, les logs d'audit et les limites de débit ;
- exploiter des données antenne plus complètes pour rendre les simulations tilt et voisins plus réalistes.

Conclusion générale

NetVision fournit un cockpit opérationnel pour comprendre l'état d'un réseau radio mobile, localiser les zones sensibles, analyser les KPI d'une cellule et évaluer des actions possibles. Sa contribution technique réside dans l'intégration d'un frontend cartographique, de données runtime locales, de routes API, d'un backend optionnel de recommandations, d'une file de jobs et d'un moteur ns-3 asynchrone.

La contribution RAN est également claire : le projet ne réduit pas la congestion à un seul indicateur. Il combine PRB, débit, CQI, utilisateurs actifs, TA et récurrence temporelle pour produire une lecture plus utile à l'ingénieur radio. La solution actuelle est déjà exploitable pour l'exploration, la démonstration technique et l'aide à la décision locale. Elle doit toutefois poursuivre son travail de calibration, de validation terrain et de durcissement production avant d'être considérée comme un jumeau numérique radio complet.



Commandes de reproduction

```
npm install
npm run dev
npm run backend
npm run worker
npm run test:contracts
npm run build
npm run ns3:check
```

Fichiers principaux inspectés

Fichier ou dossier	Rôle
pages/index.js	Point d'entrée Next.js du dashboard.
src/main.js	Composition principale du cockpit opérateur.
src/components/ src/admin/	Composants UI : carte, panels, chronologie, KPI. Agrégations, recherche, import et logique administrative.
pages/api/	Routes API Next.js pour données, jobs, santé et exports.
backend/	Service FastAPI et règles de recommandations.
job-workers/	Worker BullMQ et suivi SQLite des jobs.
simulation/ns3/ runtime_data/	Adaptateur, générateur de scénario et runner ns-3. Données runtime réelles locales.
runtime_data_mock/	Données mock étendues pour QA et démonstration.